Research Activity Report

Program	2026 SKKU ISS Research Internship
Duration	July 27, 2026 – August 20, 2026
Research Title	Exploring Grounding DINO as a Face-Localization Stage for a Deepfake-Detection Pipeline
Laboratory / Supervisor	SKKU Information Laboratory (InfoLab) / Prof. Tamer Abuhmed
Student Name / ID Number	James Corino / 2026319644
Home University	University at Albany, State University of New York

Table of Contents

I. Research Internship Overview

1.1 Introduction of the Laboratory

1.2 Assigned Research Topic and Role

1.3 Internship Goals

II. Weekly Activity Report

2.1 Week 1 (July 27 – July 31): Onboarding, Model Study, and Environment Setup

2.2 Week 2 (August 3 – 7): Batch Pipeline, Prompt Experiments, and Threshold Sweep

2.3 Week 3 (August 10 – 14): Deliverables, Supervisor Feedback, and Dataset Access

2.4 Week 4 (August 17 – 20): Expanded Experiments and Final Reporting

2.5 Seminars and Meetings Attended

2.6 Details of Experiments, Data Analysis, and Programming

III. Contribution to Research Project

3.1 Key Tasks Performed

3.2 Problem-Solving Process

3.3 Tools, Software, and Data Used

IV. Results and Achievements

4.1 Outcomes

4.2 Summary of Presentations and Assignments

V. Reflection and Future Plan

5.1 Lessons Learned

5.2 Feedback and Suggestions

5.3 Future Academic and Career Plans

VI. Appendix

A. Repository Structure

B. Batch-Inference Script Usage

C. Consolidated Experimental Data

D. References

I. Research Internship Overview

1.1 Introduction of the Laboratory

I completed my internship at the SKKU Information Laboratory (InfoLab), directed by Professor Tamer Abuhmed within the Department of Computer Science and Engineering, College of Computing and Informatics, at Sungkyunkwan University. InfoLab conducts research on artificial intelligence and information security, and my placement fell under the laboratory's "AI for Social Good" research direction, which applies modern machine-learning methods to problems with clear societal impact. The specific problem area I was assigned to, deepfake detection, is representative of this mission: as generative models make synthetic and manipulated media easier to produce, reliable automated detection has become an important safeguard for information integrity.

Work in the lab is organized around written progress reports to Professor Abuhmed, progress meetings as scheduled, and continuous documentation of all experiments in a version-controlled repository. Senior lab members (Firuz and Abdenour) were available as points of contact for technical questions throughout the internship. The lab places explicit emphasis on documenting unsuccessful experiments as carefully as successful ones, on the grounds that failure cases often determine the next research direction; this principle shaped how I kept my daily logs and reports.

1.2 Assigned Research Topic and Role

My assigned topic was the exploration of Grounding DINO, an open-vocabulary object detection model published by IDEA Research, as a candidate first-stage component in a deepfake-detection pipeline. The laboratory's longer-term objective is to investigate whether Grounding DINO can locate faces or relevant facial regions in an image before a dedicated forensic manipulation-detection model is applied downstream. My role as a research intern was to carry out the initial feasibility study: understand the model at the level of its paper and implementation, stand up a working environment, run systematic experiments on face imagery under varied conditions, characterize the model's behavior and failure modes, and report the findings in a form the lab could act on.

The assignment was structured around five concrete tasks, given to me by Professor Abuhmed at the start of the internship: (A) understand and summarize the model; (B) set up the official repository and reproduce the reference inference example; (C) conduct initial experiments on at least 20 images spanning different face conditions with five specified text prompts; (D) analyze the effect of the box-threshold and text-threshold settings; and (E) develop a batch-inference script that processes an image folder and records annotated images, predictions, confidence scores, and

inference times. In parallel, I was asked to complete a foundational PyTorch course to build the deep-learning skills the lab's subsequent work would require.

1.3 Internship Goals

From the assignment above, I set the following goals for the four-week internship:

- Develop a working understanding of open-vocabulary object detection, and of Grounding DINO's architecture, inputs, outputs, and thresholding behavior in particular.
- Build and document a reproducible experimental environment, including every installation error encountered and its resolution, so that lab members can replicate the setup.
- Produce an empirical characterization of Grounding DINO's face-detection behavior across realistic conditions (frontal, multiple, profile, occluded, small/low-resolution, and deepfake-derived imagery) sufficient to support a go/no-go decision on using it as a pipeline front end.
- Deliver working, documented tooling (a batch-inference script and analysis utilities) that the lab can reuse for follow-up experiments.
- Practice the discipline of research communication: daily logs, weekly progress reports, a findings report, and a short presentation.
- Complete the assigned PyTorch for Deep Learning course far enough to be productive in the lab's PyTorch-based codebases.

Ⅱ. Weekly Activity Report

2.1 Week 1 (July 27 – July 31): Onboarding, Model Study, and Environment Setup

The internship began with an onboarding meeting with Professor Abuhmed on Tuesday, July 28 (10:00 KST), in which the research topic, the five tasks, the weekly reporting cycle, and the evaluation criteria were laid out. The same day I read the Grounding DINO paper and the official repository documentation and drafted a written summary (Task A) covering the problem the model addresses, its image and text inputs, its outputs, the roles of the box and text thresholds, and how it differs from conventional fixed-class detectors. I also began the PyTorch course, completing Section 1.

On July 29 I set up the working environment (Task B) on my development machine, an Apple Silicon MacBook: a dedicated conda environment with Python 3.10, PyTorch 2.13.0, and the Grounding DINO repository installed in editable mode. Because Apple Silicon provides no CUDA support, I evaluated the Metal (MPS) backend, confirmed it was available, but deliberately chose CPU-only inference after finding that Grounding DINO's MPS path has known operator-support issues (torch.roll unsupported without a fallback flag; cumsum lacking int64 support, which affects accuracy) while CPU mode benchmarks comparably in community tests. Five distinct installation errors were encountered and resolved, most notably a pip build-isolation failure requiring the --no-build-isolation flag and a transformers API incompatibility requiring a pin to version 4.37, and each error and fix was recorded in the daily log. I downloaded the pretrained Swin-T checkpoint and continued the PyTorch course (Section 2).

On July 31 I successfully ran the official single-image inference demo end-to-end with the prompt "person" at default thresholds, producing a correctly annotated output image and confirming that the environment, checkpoint, and pipeline all functioned. This met the supervisor's milestone of completing setup and basic inference within the first three working days.

2.2 Week 2 (August 3 – 7): Batch Pipeline, Prompt Experiments, and Threshold Sweep

On August 3 I built the batch-inference script (Task E), src/batch_inference.py, wrapping the library's load_model, load_image, predict, and annotate functions in a loop over an input folder, with command-line arguments for the input and output folders, prompt, both thresholds, and a CPU-only flag. The script saves annotated images and a predictions.json file recording, per image, the bounding boxes, matched phrases, confidence scores, category (inferred from the subfolder name), and inference time. I debugged two device-related API errors (described in Section 3.2), then validated the script with a smoke test and a set of deliberate edge cases: an empty input folder, non-image files mixed into the folder, a mismatched prompt expected to produce zero detections,

and an overwrite check on the output folder. The same day I curated the initial test-image set (Task C): ten single frontal faces from the LFW dataset (via the Hugging Face mirror logasja/lfw after the official host proved unresponsive), plus stock photographs from Unsplash and Pexels for the multiple-face, profile, and occluded categories.

On August 4 I ran the main prompt experiments: all five specified prompts ("human face", "person", "face . eyes . mouth", "real face", "manipulated face") across the full 21-image set, one output folder per prompt, followed by the threshold sweep (Task D) on three representative images at three box/text threshold combinations (0.25/0.20, 0.35/0.25, 0.45/0.30). I wrote two small analysis utilities, summarize_results.py and compare_thresholds.py, to consolidate box counts and scores across runs. The key findings, prompt-behavior patterns and the absence of any universal threshold, are presented in Section Ⅳ.

August 5–7 was spent turning the experiments into deliverables: the initial findings report (8 pages, with figures and tables), a 6-slide presentation summarizing the findings, and continued PyTorch coursework on August 7.

2.3 Week 3 (August 10 – 14): Deliverables, Supervisor Feedback, and Dataset Access

On August 10 I wrote the weekly progress report, performed a final review of the repository (implementation, README, annotated images, prediction files, report, and presentation), and pushed the complete initial-findings package to GitHub. Professor Abuhmed reviewed the report during the week and returned written feedback: continue following up on the pending FaceForensics++ dataset request without letting it block progress; investigate two deepfake datasets from the DASH Lab (RWDF-23 and FakeAVCeleb) as alternatives; expand the thin occluded and multiple-face categories; add the missing small/low-resolution condition; and keep documenting prompts, thresholds, detections, and failure cases carefully. No meeting was held that week, so the feedback cycle was asynchronous.

On August 14 I acted on the feedback. I researched both suggested datasets and found that neither offers instant access: RWDF-23 is gated by a Google Form request and FakeAVCeleb by a form plus a license agreement. I submitted both requests the same day to start the approval clocks, and sent a follow-up on the still-pending FaceForensics++ request. To avoid blocking on approvals, I identified an openly downloadable Kaggle deepfake-image sample (manjilkarki/deepfake-and-real-images) as a clearly labeled, non-forensic placeholder. I also created the new small/low-resolution category: six images generated by a reproducible script (src/make_small_lowres.py) that downscales already-curated photographs to 32–90 pixels on the long edge, deliberately isolating resolution as the only changed variable. Finally, I curated a shortlist of candidate stock photographs for expanding the occluded and multiple-face categories.

2.4 Week 4 (August 17 – 20): Expanded Experiments and Final Reporting

On August 17 I completed the data expansion (occluded grew from 2 to 7 images with masks, hands, hair, sunglasses-and-scarf combinations, and partial crops; multiple-faces grew from 4 to 5 with an additional dense crowd scene) and re-ran the batch pipeline with all five standard prompts across the occluded, multiple, small/low-resolution, and (still empty, as a pipeline check) deepfake categories, twenty runs in total, spot-checking annotated outputs per category. This produced the most significant finding of the internship: the new dense crowd photograph, containing hundreds of people with faces roughly 30–40 pixels tall, returned zero detections on four of the five prompts, a qualitatively different failure mode from the duplicate-box behavior observed in Week 2.

On August 18 I downloaded the 12-image Kaggle deepfake sample into data/deepfake/ and ran the five standard prompts against it, then ran the full threshold sweep on the occluded, multiple, and small/low-resolution folders (nine runs), confirming that the crowd-density failure is not fixable by threshold tuning: even the most permissive setting recovered only two very-low-confidence boxes out of hundreds of visible faces. Along the way I found and fixed a shell-portability bug that had silently failed all nine sweep runs (zsh word-splitting differs from bash; described in Section 3.2). I then built the second findings report (Report v2, 8 pages) covering all new categories and results, made a final README pass, and updated the progress report to close out the internship.

The final days of the internship (August 19–20) were devoted to consolidating the repository, completing this Research Activity Report and the final presentation materials, and continuing the PyTorch course, which I completed through 10 of its 14 sections by the end of the internship.

2.5 Seminars and Meetings Attended

Supervision during the internship took place through three main touchpoints with Professor Abuhmed, supplemented throughout by written documentation (daily logs and progress reports) in the shared repository:

- July 28: Onboarding meeting with Professor Abuhmed (10:00 KST): research topic, task list, reporting expectations, and evaluation criteria.
- August 10: Two-week progress review. I submitted the complete initial-findings package (findings report, presentation, repository, and a written progress report covering tasks completed, results and code updates, problems and unsuccessful attempts, plans, and support needed). Professor Abuhmed reviewed it and returned detailed written feedback during the week, which directly set the Week 3–4 experimental agenda.

- August 20: Final report. This Research Activity Report, submitted together with the v2 findings report and the final state of the repository, serves as the closing deliverable and review point of the internship.
- Technical contact points: Senior lab members Firuz and Abdenour were designated for technical questions; the crowd-density zero-detection finding was flagged in my final progress report as a topic for a sanity check with them.

2.6 Details of Experiments, Data Analysis, and Programming

All experiments were run with the pretrained Grounding DINO Swin-T checkpoint in CPU-only mode. The experimental program had four components, each documented day-by-day in the repository's logs:

- Prompt experiments (Task C). Five prompts were run across every image category, one output folder per prompt so that predictions were never overwritten. Box counts, matched phrases, and confidence scores were consolidated per category and per prompt with summarize_results.py. Across full runs, inference averaged 3.37 seconds per image on the Apple M2 in CPU-only mode.
- Threshold analysis (Task D). Representative images were re-run at box/text threshold combinations 0.25/0.20, 0.35/0.25, and 0.45/0.30, first on a three-image subset in Week 2 and then across the full occluded, multiple, and small/low-resolution categories in Week 4, with results compared using compare_thresholds.py and confirmed visually against the annotated outputs.
- Dataset engineering. The test set grew from 21 images in four categories to 45 images in six categories over the internship. The small/low-resolution category was generated programmatically from existing curated images to isolate resolution as a variable, and the deepfake category used a clearly labeled Kaggle sample pending approval of the three requested research datasets (FaceForensics++, RWDF-23, FakeAVCeleb).
- Programming. Beyond the batch-inference script, I wrote the two analysis utilities and the low-resolution generator, and validated edge-case behavior (empty folders, non-image files, mismatched prompts, output-overwrite semantics) before trusting the pipeline for experiments.

Ⅲ. Contribution to Research Project

3.1 Key Tasks Performed

My contribution to the lab's deepfake-detection research line can be summarized as the complete initial feasibility study for the proposed Grounding DINO front end, delivered as a documented, reproducible package:

- Model summary (Task A). A written summary of Grounding DINO covering the open-set detection problem, the image/text dual inputs, the fused vision–language pipeline, the box-and-phrase outputs, the semantics of the box and text thresholds, and the contrast with fixed-vocabulary detectors.
- Reproducible environment (Task B). A fully documented macOS/Apple Silicon setup, including the reasoned choice of CPU-only inference over MPS, every installation error with its fix, and a README that allows any lab member to replicate the environment, including on CUDA machines, for which the README notes the alternative install path.
- Curated benchmark set (Task C). A 45-image, six-category face test set (single frontal, multiple, profile/angled, occluded/blurred, small/low-resolution, deepfake sample) with documented sourcing and licensing-conscious curation, plus a reproducible generator for the low-resolution condition.
- Threshold characterization (Task D). An empirical account of how box/text threshold settings trade missed faces against duplicate and false detections across scene types, including the finding that no single global threshold is adequate.
- Reusable tooling (Task E). The batch-inference script with JSON prediction output and per-image timing, plus the two analysis utilities, all exercised across roughly forty recorded experiment runs.
- Failure-mode discovery. Identification and threshold-sweep confirmation of the dense-crowd zero-detection failure mode, now written up as its own section of the v2 findings report as a candidate driver of the next research direction (e.g., image tiling before inference).
- Reporting. Two findings reports, a presentation, weekly progress reports, fourteen daily logs, and this activity report.

3.2 Problem-Solving Process

The internship's working principle was to treat every obstacle as something to be diagnosed, fixed, and documented rather than worked around silently. The table below lists the principal technical problems encountered and how each was resolved.

#	Problem	Diagnosis and Resolution
1	No CUDA support on Apple Silicon; Grounding DINO's MPS backend has known operator issues (torch.roll, int64 cumsum)	Verified MPS availability but selected CPU-only inference for reliability and accuracy, after confirming CPU mode benchmarks comparably in community tests; documented the trade-off
2	pip install -e . failed inside pip's isolated build environment (ModuleNotFoundError: no torch)	Re-ran with --no-build-isolation so the build used the torch already present in the active conda environment
3	AttributeError: 'BertModel' object has no attribute 'get_head_mask' at model load	Version mismatch: newer transformers removed an API the model code depends on; pinned transformers==4.37
4	TypeError: load_model() got an unexpected keyword argument 'cpu_only'	Inspected the real function signature with inspect.signature and switched to the correct device parameter
5	AssertionError: Torch not compiled with CUDA enabled inside predict(), even after fixing load_model	Found that predict() has its own independent device parameter defaulting to "cuda"; passed device="cpu" explicitly and grepped the source to confirm no other silent defaults
6	Official LFW download host unresponsive	Switched to the Hugging Face mirror (logasja/lfw) via the datasets library
7	All nine Week-4 threshold-sweep runs failed silently with an argparse invalid-float error	Traced to zsh word-splitting semantics differing from bash (set -- $combo does not split); replaced with portable parameter-expansion splitting and re-ran successfully
8	Gated access to all three forensic datasets (FaceForensics++, RWDF-23, FakeAVCeleb)	Submitted all requests promptly, followed up on the oldest, and unblocked experiments with a clearly labeled non-forensic Kaggle placeholder in the interim

Two habits made this process effective. First, reading the underlying source rather than assuming API consistency: problems 4 and 5 arose precisely because the demo script's interface differs from the library's, and were solved by direct inspection. Second, validating tooling on deliberate edge cases before running real experiments, which is why the empty-folder behavior later encountered in Week 4 (an as-yet-unfilled deepfake folder) was already known to be safe.

3.3 Tools, Software, and Data Used

Category	Details
Hardware	Apple MacBook (M2 chip), macOS 26.6; CPU-only inference (no

Category	Details
	NVIDIA GPU; MPS available but not used)
Languages / runtimes	Python 3.10 (conda environment via miniforge)
Core libraries	PyTorch 2.13.0, torchvision, transformers 4.37 (pinned), Grounding DINO (official IDEA-Research repository, editable install), Pillow, Hugging Face datasets
Model	Grounding DINO Swin-T, official pretrained checkpoint (groundingdino_swint_ogc.pth)
Own tooling	src/batch_inference.py, src/summarize_results.py, src/compare_thresholds.py, src/make_small_lowres.py
Data	LFW via Hugging Face mirror (single frontal); Unsplash/Pexels stock photography (multiple, profile, occluded); programmatic downscales (small/low-res); Kaggle manjilkarki/deepfake-and-real-images, Test/Fake split (deepfake placeholder)
Requested datasets	FaceForensics++, RWDF-23, FakeAVCeleb (DASH Lab); access requests submitted, pending at internship end
Infrastructure	Git/GitHub (version control, daily commits), Markdown daily logs, Udemy PyTorch for Deep Learning course

IV. Results and Achievements

4.1 Outcomes

Test set. The final benchmark comprised 45 images across six categories:

Category	Count	Source
Single frontal face	10	LFW (Hugging Face mirror logasja/lfw)
Profile / angled face	5	Stock photos (Unsplash / Pexels)
Multiple faces	5 (from 4)	Stock photos, incl. one dense crowd scene added in Week 4
Occluded / blurred face	7 (from 2)	Stock photos: masks, hands, hair, sunglasses + scarf
Small / low-resolution	6	Programmatic downscales of curated images (32–90 px long edge)
Deepfake sample	12	Kaggle deepfake-and-real-images (non-forensic placeholder)

Prompt behavior. The five-prompt comparison produced four consistent findings. First, "face . eyes . mouth" reliably multiplied box counts (3–11 boxes per single-face image versus 1–4 for "human face") because the dot-separated prompt creates three distinct grounding targets, not because of higher sensitivity. Second, the exploratory forensic prompts "real face" and "manipulated face" tracked plain "human face" almost exactly; for example, per-image counts on the multiple-face category were [18, 3, 5, 6] (human) versus [17, 3, 5, 6] (real), indicating the model matches on "face" and does not respond meaningfully to the real/manipulated qualifier. The pattern replicated on the deepfake sample in Week 4 (average boxes per image: human face 1.33, real face 1.25, manipulated face 1.17), confirming the supervisor's caveat that these prompts carry no reliable forensic signal. Third, "person" and "human face" are not interchangeable: identical on profile and occluded images but divergent with scene complexity (10 versus 18 boxes on the densest Week-2 crowd; 2.3 versus 1.6 average boxes on single faces). Fourth, detection was stable at one box per image on profile and occluded faces, a robustness the Week-4 expansion to seven occluded images substantiated (six of seven at exactly one box).

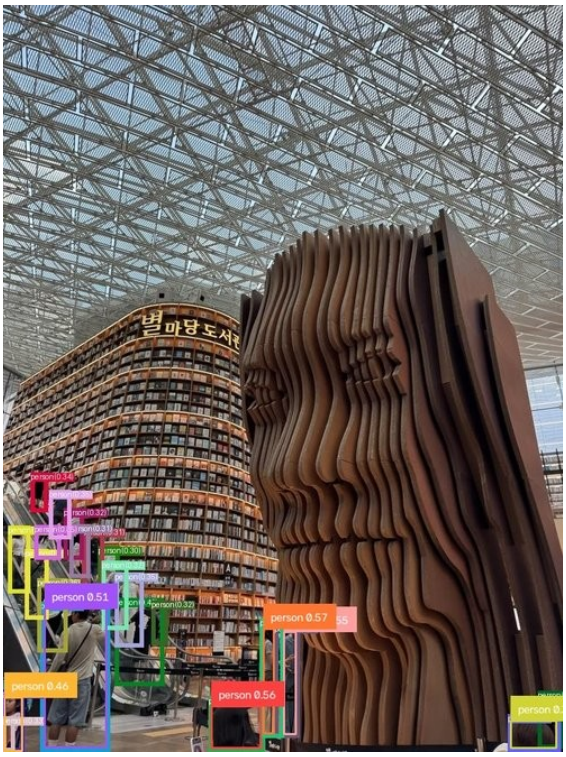

Figure 1. Example annotated output from the batch-inference script (prompt "person", default thresholds).

Threshold sensitivity. No single threshold setting performed well across all conditions. The Week-2 sweep on three representative images showed that raising the threshold removes genuine faces, not merely noise: background faces in the frontal image, most of a crowded scene, and an occluded face entirely.

Image (category)	0.25 / 0.20	0.35 / 0.25	0.45 / 0.30	Interpretation
lfw_02 (single)	4 boxes (0.55–0.36)	4 boxes, identical	1 box (0.55)	Extra boxes at lower thresholds are genuine background faces, not duplicates; 0.45 drops real detections
nicholas-green (multiple)	26 boxes	18 boxes	1 box	Heavy duplication at low threshold; 0.45 collapses a genuinely crowded scene to one face
pexels-moh-adbelghaffar (profile/occluded)	3 boxes (0.40–0.26)	1 box (0.40)	0 boxes	Highest threshold misses the face entirely

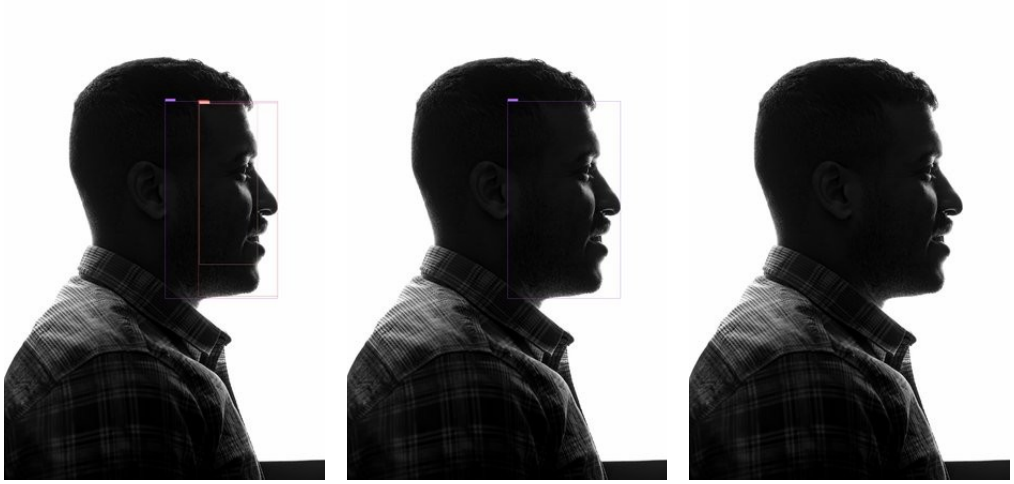

Figure 2. The profile/occluded image across the threshold sweep (0.25/0.20, 0.35/0.25, 0.45/0.30): three boxes, one box, then a complete miss.

Figure 3. The Week-2 crowd image across the same sweep: 26 → 18 → 1 boxes.

A control run with a deliberately mismatched prompt ("bicycle") produced zero detections on a face image, indicating that the additional boxes surfaced at lower thresholds correspond to face-like content rather than random false positives. The library default of 0.35/0.25 proved a reasonable general-purpose middle ground, but the practical conclusion for the pipeline is that a production system needs either a per-scene-type threshold policy or post-processing such as non-maximum-suppression tuning; a stricter global cutoff is not a substitute, because it also removes genuine detections.

Dense-crowd failure mode. The most significant single finding came from the Week-4 crowd scene (hundreds of people, faces roughly 30–40 px): zero detections on four of five prompts at default thresholds, versus 18 boxes on the Week-2 crowd outlier under the same prompt. The threshold sweep confirmed this is not a tuning problem: the most permissive setting (0.25/0.20) recovered only two boxes at confidences of 0.296 and 0.264 out of hundreds of visible faces. By contrast, single-subject thumbnails down to 32–56 px were still detected at one box each with confidence comparable to full resolution, and 90-px multi-subject thumbnails degraded only partially (4–5 faces still found). Face scale interacting with scene density, rather than low resolution alone, therefore appears to drive the failure, a concrete and actionable limitation for the planned pipeline, with image tiling before inference identified as the follow-up direction.

Overall assessment. For the lab's purpose, the study supports a qualified positive conclusion: Grounding DINO localizes faces reliably across pose, moderate occlusion, and moderate resolution loss, making it a plausible first-stage face localizer; but its raw output requires post-processing, its thresholds require scene-aware policy, its prompts carry no forensic signal whatsoever, and dense small-face crowds are currently out of reach without additional measures.

4.2 Summary of Presentations and Assignments

- Grounding DINO Initial Findings report (8 pages, Week 2–3): model summary, environment, methodology, prompt and threshold results with figures, limitations, challenges, next steps.
- Initial Findings presentation (6 slides): motivation and pipeline role, experiment design, prompt behavior, threshold sensitivity, limitations and next steps.
- Findings Report v2 (8 pages, Week 4): standalone update covering the expanded occluded/multiple categories, the new small/low-resolution and deepfake-sample results, and the dense-crowd failure-mode analysis.
- Written progress reports at each review point, covering completed tasks, results and code updates, problems and unsuccessful attempts, plans, and support needed.
- Fourteen daily logs in a fixed template (to-do, experiments, results, challenges with fixes, next steps), committed to the repository.
- PyTorch for Deep Learning (Udemy): 10 of 14 sections completed by internship end, including the fundamentals, workflow, classification, and computer-vision material, with exercises logged alongside the daily entries.

V. Reflection and Future Plan

5.1 Lessons Learned

Technically, the internship taught me how open-vocabulary detection actually behaves outside its benchmarks. The most valuable lessons were empirical: that a prompt's structure can change output semantics (dot-separated prompts create separate grounding targets); that threshold settings encode a genuine trade-off with no universal optimum, so "tuning" must be scene-aware; and that a model can degrade gracefully along one axis (resolution) while failing abruptly along another (face scale in dense scenes). I also learned to distrust surface-level API consistency (two of my hardest bugs came from a demo script and its underlying library disagreeing about device handling) and to respect environment engineering as real research work: the transformers pin, the build-isolation flag, and the MPS/CPU decision each cost investigation and each now saves the next lab member that same cost.

Methodologically, I internalized the lab's documentation discipline. Writing down every failed run, including a shell bug that silently invalidated nine experiments, turned out to be what made the final reports trustworthy: I could state exactly which results came from which runs at which settings. The weekly reporting rhythm (tasks, results, failures, plans, support needed) forced me to convert raw activity into findings continuously rather than at the end. And the experience of having a supervisor's written feedback reshape my experimental agenda mid-internship (Week 3) showed me how the feedback loop between advisor and student actually drives a research project forward.

Practically, I learned how much of applied research is unblocked by anticipating dependencies: submitting all three dataset-access requests early and preparing a clearly labeled placeholder meant that gated data never stalled the experimental program.

5.2 Feedback and Suggestions

The internship was well structured: a concrete task list from day one, an explicit three-day milestone for environment setup, weekly written reporting, and fast, specific supervisor feedback that always translated directly into next experiments. The instruction to document failures as carefully as successes was the single most formative expectation, and I would recommend keeping it central to future iterations of the program.

Two suggestions from a participant's perspective. First, dataset access is the natural bottleneck for forensic-media research: future interns in this line would benefit from having access requests to FaceForensics++ (or the lab's own preferred datasets) initiated before or at the very start of the internship, since approval latency consumed a large share of a four-week program; all three of my requests were still pending at internship end. Second, a brief early introduction to the lab's senior

members and their areas (beyond names as contact points) would lower the barrier to asking technical questions earlier; I flagged the crowd-density finding for their review, but a mid-internship technical check-in might have surfaced that conversation sooner.

5.3 Future Academic and Career Plans

This internship confirmed my intention to pursue research in machine learning, and in trustworthy AI in particular. In the near term, I plan to complete the remaining sections of the PyTorch course and continue contributing to this project's next phase where possible: the immediate open items (re-running the forensic-prompt comparison on genuine manipulated frames once dataset access is granted, and investigating tiling-based approaches to the dense-crowd failure) are well defined, and I would welcome the chance to pursue them with the lab remotely during the academic year.

As a concrete first step, I plan to meet with Professor Abuhmed to discuss next steps: whether and in what form I can stay involved in the project's next phase, which of the open items above make sense for me to take on, and what continued collaboration with the lab could look like going forward. Longer term, I am planning to apply to graduate programs in computer science with a focus on AI, and the experience of taking a research question from a paper to an empirical, documented, decision-ready answer, under a real lab's standards for rigor and reporting, is the model of the work I want to continue doing. The societal framing of the lab's mission also stays with me: deepfake detection is a concrete case where careful engineering serves the public interest, and it is the kind of problem I hope to keep working on.

VI. Appendix

A. Repository Structure

All work is version-controlled in the internship repository (github.com/Jamescorino8/InfoLab-Research-Internship). Layout:

```
.
├── README.md           # setup, usage, image-set documentation
├── Day01/ ... Day14/   # daily logs (+ per-day materials)
├── GroundingDINO/      # cloned official repository
├── weights/            # pretrained checkpoint (gitignored)
├── src/                # batch_inference.py, summarize_results.py,
│                       # compare_thresholds.py, make_small_lowres.py
├── data/               # test images: single/ profile/ multiple/
│                       # occluded/ small_lowres/ deepfake/
│                       # subsets_for_thresholds/
├── results/            # annotated images + predictions.json per run
└── reports/            # findings reports v1-v2, presentation,
                        # weekly reports
```

B. Batch-Inference Script Usage

The batch-inference script accepts the input folder, prompt, and thresholds as command-line arguments and writes annotated images, a predictions.json (boxes, phrases, confidence scores, per-image inference time), inferring each image's category from its subfolder:

```
python src/batch_inference.py \
  --input_folder data \
  --output_folder results/prompt_human_face \
  --prompt "human face" \
  --box_threshold 0.35 --text_threshold 0.25 \
  --cpu_only
```

C. Consolidated Experimental Data

Per-image box counts by prompt for the Week-2 run (default thresholds, 0.35/0.25):

Category	“human face”	“person”	“face . eyes . mouth”	“real face”	“manipulated face”
Single (10)	[1,1,4,1,1,1,2,1,2,2]	avg 2.3 boxes	[5,7,4,3,7,5,7,5,11,8]	tracks human face	tracks human face
Multiple (4)	[18,3,5,6]	10 on densest scene	elevated (3 targets)	[17,3,5,6]	[3,2,5,6]
Profile (5)	1 box/image	1 box/image	elevated (3	1 box/image	1

Category	“human face”	“person”	“face . eyes . mouth”	“real face”	“manipulated face”
			targets)		box/image
Occluded (2)	1 box/image	1 box/image	elevated (3 targets)	1 box/image	1 box/image

Deepfake-sample run (Week 4, 12 images, default thresholds), average boxes per image: human face 1.33; person 1.58; face . eyes . mouth 4.58; real face 1.25; manipulated face 1.17 (including the sample's only zero-detection). Small/low-resolution run: single-subject thumbnails (32–56 px) detected at 1 box each at near-full-resolution confidence; multi-subject thumbnails (90 px) detected 4–5 faces at reduced count and confidence. Dense-crowd image: 0 detections on 4 of 5 prompts at defaults; 2 boxes (0.296, 0.264) at 0.25/0.20.

D. References

- Liu, S., et al. “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection.” IDEA Research. Official repository: github.com/IDEA-Research/GroundingDINO
- Rössler, A., et al. “FaceForensics++: Learning to Detect Manipulated Facial Images.” (dataset access requested)
- “Towards Understanding of Deepfake Videos in the Wild” (RWDF-23), CIKM 2023. arxiv.org/abs/2309.01919 (dataset access requested)
- Khalid, H., et al. “FakeAVCeleb: A Novel Audio-Video Multimodal Deepfake Dataset.” DASH Lab. github.com/DASH-Lab/FakeAVCeleb (dataset access requested)
- LFW (Labeled Faces in the Wild), via Hugging Face mirror logasja/lfw.
- Kaggle: manjilkarki/deepfake-and-real-images (placeholder sample).
- Internship repository: github.com/Jamescorino8/InfoLab-Research-Internship